Basics Of C

Introduction....2
What is a Token?....2
1. Keywords....2
2. Identifiers....7
3. Constants....8
4. Operators....10
5. Special Symbols....12
DATA TYPES IN C....12
Classification of Data Types....13
1. Primary (Basic) Data Types....14
2. Derived Data Types....15
3. User-Defined Data Types....15
Modifiers in C....16
Qualifiers in C....17
What is a Format Specifier?....19
Variable in C....22
First C Program – Hello World....28
What is a Program?....28
Types of Instructions in C....39
1. Type Declaration Instructions....39
2. Arithmetic Instructions....39
3. Control Instructions....39
4. Input/Output Instructions....40
Input....40
Output....40
Operators in C....44
Classification of Operators....45
1. Arithmetic Operators....46
2. Relational Operators....47
3. Logical Operators....50
7. Increment and Decrement Operators....65
Increment Operator (++)....65
Decrement Operator (--)....66
7. Conditional (Ternary) Operator....72
8. sizeof Operator....78
9. Comma Operator....85

11. Address Operator (&)..... 86
12. Pointer Operator (*)..... 86
Operator Precedence (Important)..... 86
Practice Question:..... 87

Introduction

This chapter covers the fundamental concepts of the C programming language. It provides all the basic knowledge required to begin learning and developing programs in C.

What is a Token?

A token is the smallest individual unit of a C program that is meaningful to the compiler.

Types of Tokens in C

- Keywords
- Identifiers
- Constants
- String Literals
- Operators
- Special Symbols

1. Keywords

Definition:
Keywords are reserved words in C language that have predefined meanings and are used to perform specific tasks in a program.

Important Points:

- Keywords cannot be used as identifiers (names)
- Their meaning is fixed by the compiler
- Total 32 keywords in traditional C (like int, float, if, while, etc.)

Examples:

```
int a;
if(a > 0)
```

Here:

- int, if are keywords

ANSI C (C90) Keywords (32 Keywords)

Keyword	Description
`auto`	Declares an automatic (local) variable
`break`	Terminates a loop or `switch` statement
`case`	Defines a case in a `switch` statement
`char`	Character data type
`const`	Defines a constant variable
`continue`	Skips the remaining loop body and starts the next iteration
`default`	Default case in a `switch` statement
`do`	Starts a `do-while` loop
`double`	Double-precision floating-point data type

`else`	Alternative block for an `if` statement
`enum`	Defines an enumeration
`extern`	Declares a global variable or function defined elsewhere
`float`	Single-precision floating-point data type
`for`	Starts a `for` loop
`goto`	Transfers control to a labeled statement
`if`	Conditional statement
`int`	Integer data type
`long`	Long integer type modifier
`register`	Suggests storing a variable in a CPU register
`return`	Returns control from a function

`short`	Short integer type modifier
`signed`	Signed integer type
`sizeof`	Returns the size of a data type or variable in bytes
`static`	Declares a static variable or function
`struct`	Defines a structure
`switch`	Multi-way selection statement
`typedef`	Creates a new name (alias) for a data type
`union`	Defines a union
`unsigned`	Unsigned integer type
`void`	Indicates no value or no return type
`volatile`	Indicates that a variable may change unexpectedly

`while`	Starts a `while` loop

Additional Keywords in C99

C99 introduced the following keywords:

Keyword	Description
`inline`	Suggests inline expansion of a function
`restrict`	Indicates exclusive access to a pointer's referenced object
`_Bool`	Boolean data type
`_Complex`	Complex number data type
`_Imaginary`	Imaginary number data type (rarely implemented)

Additional Keywords in C11

C11 added several more keywords:

Keyword	Description

`_Alignas`	Specifies alignment requirement
`_Alignof`	Returns alignment requirement
`_Atomic`	Declares atomic objects
`_Generic`	Generic selection expression
`_Noreturn`	Indicates a function does not return
`_Static_assert`	Compile-time assertion
`_Thread_local`	Declares thread-local storage

2. Identifiers

Definition:
Identifiers are the user defined names given to variables, functions, arrays, structures, or any user-defined elements in a C program.

Important Points:

- Used to identify program elements
- Must follow naming **rules**:
    - Start with letter or _
    - Cannot start with number and special symbol
    - Cannot be a keyword
    - No special symbols except _

Rule	Valid Example	Invalid Example	Reason
Start with a letter or _	count, _temp	1count	Identifiers must begin with a letter (A–Z, a–z) or an underscore (_).
Cannot start with a number	value1, num123	123value	Digits are allowed only after the first character.
Cannot be a keyword	total, sum	int, while, return	Keywords have predefined meanings in C and cannot be used as identifiers.
No special symbols except _	student_name, roll_no1	student-name, roll@no, total$, first.name	Only letters, digits, and the underscore (_) are allowed in identifiers.

Examples:
int marks;
float total_sum;

Here:

- marks, total_sum are identifiers

3. Constants

Definition:
Constants are fixed values that do not change during the execution of a program.

Important Points:

- Value remains constant throughout program
- Cannot be modified once defined

Types of Constants:

- **Integer Constants**

  Whole numbers without decimal point
  Example:

  10, -25, 100, 100U, 100L ,100UL, 0x1A, 0xFF

Memory Size

The memory size depends on the **type selected for the integer constant** and the compiler.

Integer Type	Typical Size	Example
`int`	**4 bytes**	`100`
`unsigned int`	**4 bytes**	`100U`
`long int`	**4 or 8 bytes**	`100L`
`unsigned long`	**4 or 8 bytes**	`100UL`
`long long`	**8 bytes**	`100LL`

Suffux	Meaning	Example
`U` / `u`	unsigned	`100U`
`L` / `l`	long	`100L`
`UL`	unsigned long	`100UL`

`LL`	long long	`100LL`
`ULL`	unsigned long long	`100ULL`

- **Floating Point Constants**

  A floating-point constant is a numeric constant that contains a decimal point or an exponent**.**

  Example:

  3.14, -0.56, 3.14, 0.5, 25.0, -12.75, 0.0, 3.14F, 3.14f, 1.5e2 , 2.5E-3

  Memory Size

Constant Type	Data Type	Typical Size
`3.14`	`double`	**8 bytes**
`3.14f` / `3.14F`	`float`	**4 bytes**
`3.14L` / `3.14l`	`long double`	**Compiler-dependent**

  Important: A floating-point constant **without a suffix is `double` by default**.

  Suffixes

  - `f` / `F` → `float`
  - `l` / `L` → `long double`

  3.14 // double
  3.14F // float
  3.14L // long double

  Exponential Notation

  `e` or `E` represents a power of 10.

1.5e2 // 1.5 × 10^2 = 150
2.5E-3 // 2.5 × 10^{-3} = 0.0025

- **Character Constants**

  A **character constant** is a single character enclosed within **single quotes (' ')**.
  Example: 'A', 'b'

  An **escape sequence character constant** is a special character represented by a backslash (\) followed by one or more characters. It is used to represent non-printable characters or characters with special meanings in C.

  Escape Sequence Table

Escape Sequence	Meaning
\a	Alert (Beep)
\b	Backspace
\f	Form Feed
\n	New Line
\r	Carriage Return
\t	Horizontal Tab
\v	Vertical Tab
\\	Backslash (\)
\'	Single Quote (')
\"	Double Quote (")
\?	Question Mark (?)
\0	Null Character

Memory Size

A character constant in C has type `int`, not `char`.

Item	Typical Size
Character constant, e.g. `'A'`	**4 bytes** (`int`)
`char` variable storing `'A'`	**1 byte**

- **String Constants**

A string constant is a sequence of characters enclosed within double quotes (" "). A string constant is automatically terminated by a null character ('\0').

Where is a String Constant Stored?

A **string constant** is stored in the **read-only memory area** (also called the **String Literal Area** or **Read-Only Data Section (.rodata)**) of a program.

Example:

"Hello"

4. Operators

Definition:
Operators are symbols used to perform operations on variables and values in a program.

Important Points:

- Used for calculations, comparisons, logic
- Operate on operands.

Here is the expanded table with all common C operators:

Type	Purpose	Operators

Arithmetic Operators	Perform mathematical operations	+ - * / %
Relational Operators	Compare values	== != > < >= <=
Logical Operators	Combine conditions	&& || !
Assignment Operators	Assign values	= += -= *= /= %=
Bitwise Operators	Operate on bits	& | ^ ~ << >>
Increment / Decrement Operators	Increase or decrease value by 1	++ --
Conditional (Ternary) Operator	Shorthand for if-else	? :
Comma Operator	Evaluate multiple expressions	,
Sizeof Operator	Returns size of a variable or type	sizeof()
Pointer Operators	Work with memory addresses	* &
Member Access Operators	Access struct or union members	. ->

Type Cast Operator	Convert one type to another	(type)

---

5. Special Symbols

Definition:
Special symbols are characters used to define the structure and organization of a C program.

Important Points:

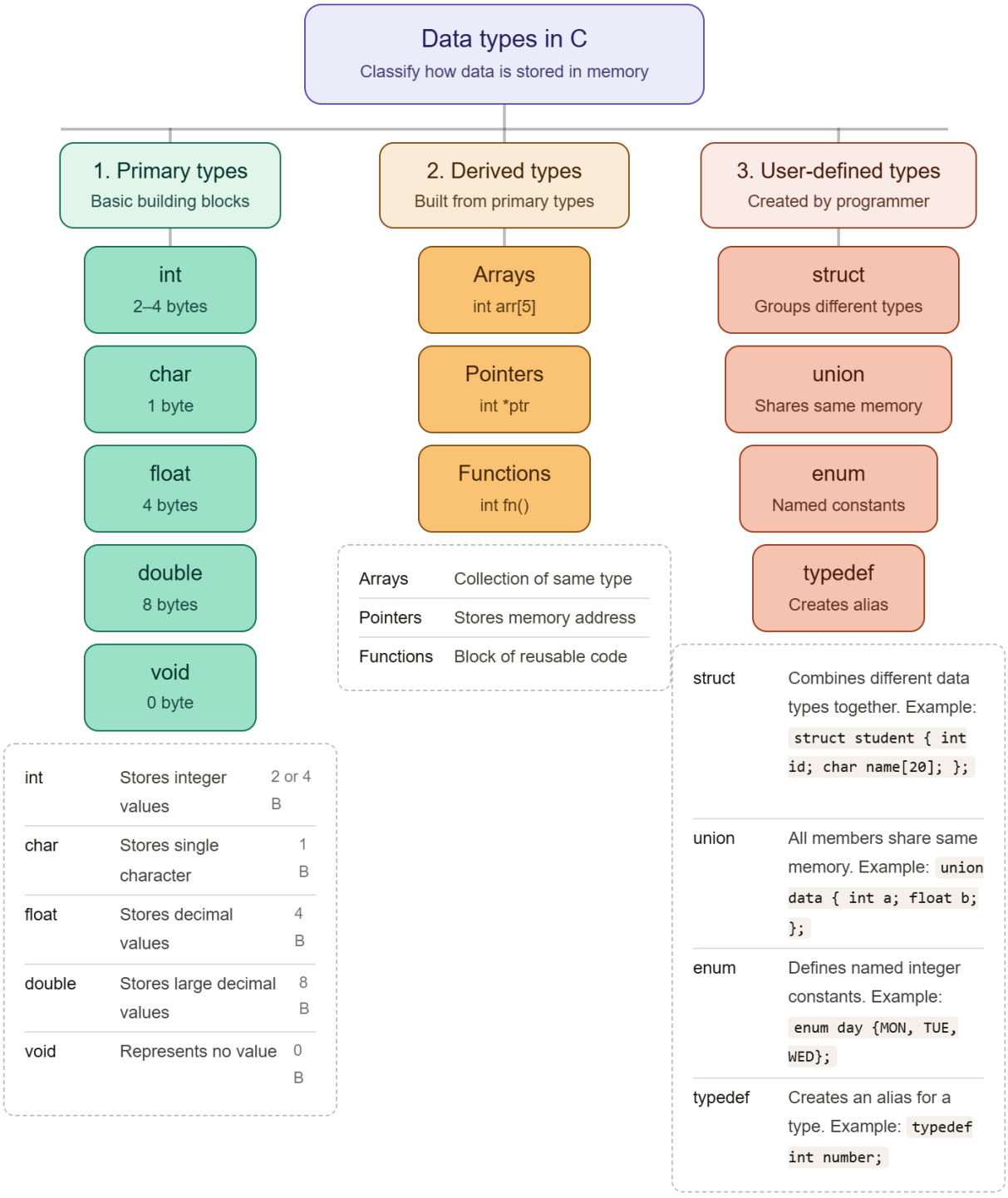
- Also called punctuators
- Help in grouping, separating, and structuring code

Examples:
`{} () [] ; , #`

DATA TYPES IN C

Definition of Data Type

A data type in C is a keyword that specifies the type of data a variable can store, the amount of memory it occupies, and the operations that can be performed on it.



Classification of Data Types

1. Primary (Basic) Data Types

Definition:

Primary data types are the basic building blocks of a C program used to store single values.

Table:

Data Type	Description	Size (Typical)
int	Stores integer values	2 or 4 bytes
char	Stores single character	1 byte
float	Stores decimal values	4 bytes
double	Stores large decimal values	8 bytes
void	Represents no value	0 byte

Why is int 2 Bytes or 4 Bytes?

The size of the int data type is not fixed by the C language. It depends on the compiler and processor architecture.

- On 16-bit systems, int is typically 2 bytes (16 bits).
- On 32-bit and 64-bit systems, int is typically 4 bytes (32 bits).

The compiler chooses the size that provides the best performance for the target processor.

System Type	Typical Size of int
16-bit System	2 Bytes
32-bit System	4 Bytes

64-bit System	4 Bytes

Note: The C standard only requires that an int be at least 16 bits in size.

2. Derived Data Types

Derived data types are the data types that are derived from primary data types to form more complex data structures.

Types:

- Arrays
- Pointers
- Functions

3. User-Defined Data Types

User-defined data types are data types created by the user to store and manage complex data efficiently.

- **Structure (struct)**

```
struct student {
  int id;
  char name[20];
};
```

- **Union (union)**

```
union data {
  int a;
  float b;
};
```

- **Enumeration (enum)**

```
enum day {MON, TUE, WED};
```

Modifiers in C

Modifiers are keywords used to modify the size, range, or sign of a data type.

Types of Modifiers:

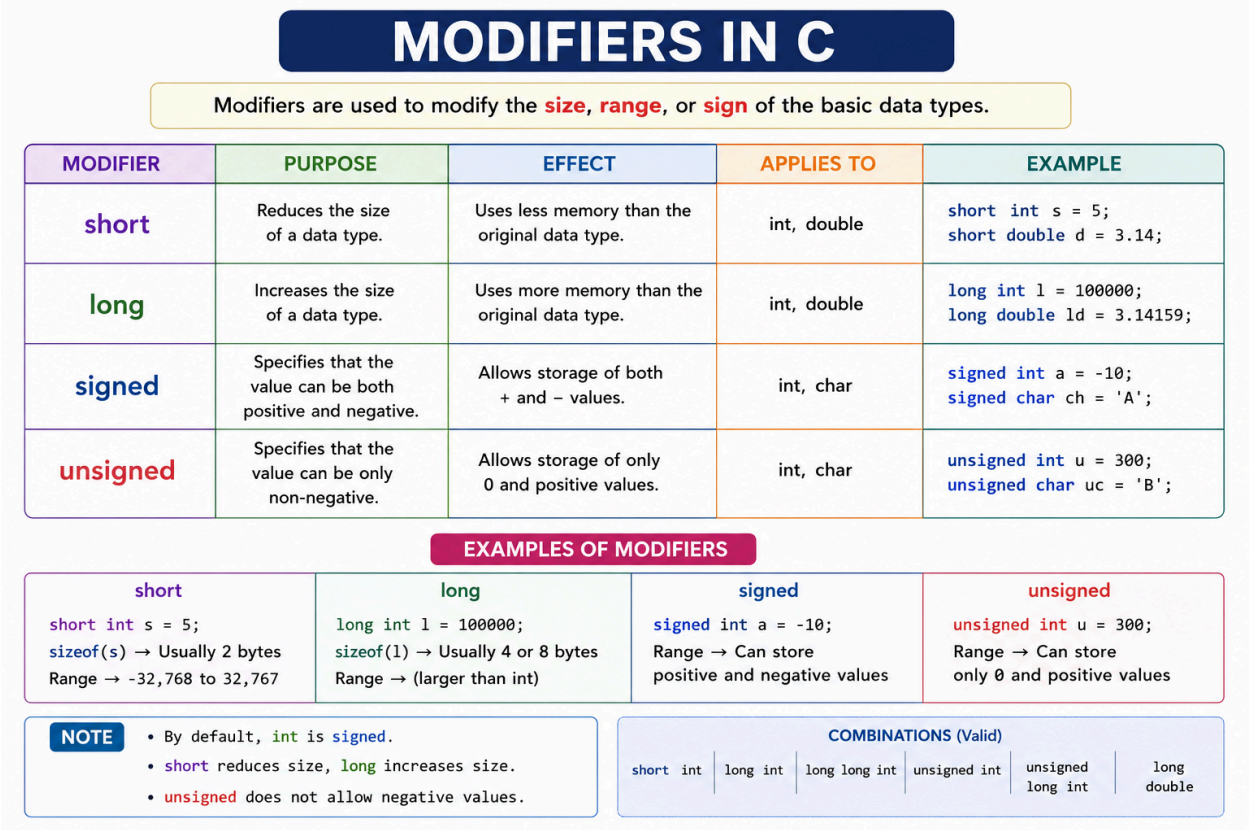
- **short**
- **long**
- **signed**
- **unsigned**

Examples:

- short int a;
- long int b;
- unsigned int c;

Purpose:

- Increase or decrease storage size
- Change range of values



Qualifiers in C

Qualifiers are keywords used to define special properties or behavior of variables.

Types:

- **const**
- **volatile**

Volatile Qualifier

Definition:

The volatile qualifier is used to tell the compiler that the value of a variable may change at any time without any action by the program.

Explanation:

- Normally, the compiler optimizes code and may assume a variable value does not change unexpectedly.
- But some variables can change due to:
    - Hardware signals
    - Interrupts
    - External devices

In such cases, volatile prevents optimization.

Syntax:

volatile int value;

Key Features:

- Prevents compiler optimization
- Forces variable to be read from memory every time
- Ensures correct behavior in embedded systems

Real-Time Example:

volatile int flag;

while(flag == 0);

Here:

- Program waits until flag changes
- If not volatile → compiler may optimize → infinite loop

- If volatile → value checked every time

Applications:

- Embedded systems
- Hardware registers
- Interrupt Service Routines (ISR)

const Qualifier in C

The **const qualifier** is used to declare a variable whose value cannot be modified after initialization.

Syntax

const data_type variable_name;

Example

const int MAX = 100;

MAX = 200; // Error

Features

- Makes a variable **read-only**.
- Prevents accidental modification of data.
- Improves program reliability and readability.
- Can be used with variables, pointers, function parameters, and arrays.

Example

const float PI = 3.14159;

printf("%f", PI);

Output

3.14159

Important Note

const int x = 10;

```
x = 20;      // Compilation Error
```

The compiler generates an error because a const variable cannot be modified after it is initialized.

Common Uses

```
const int MAX_SIZE = 100;
const char GRADE = 'A';
const float PI = 3.14;
```

What is a Format Specifier?

A format specifier is a special symbol used in functions like printf() and scanf() to define the type of data to be displayed or read.

Definition

A format specifier is a placeholder starting with % that tells the compiler what type of data is being printed or taken as input.

Syntax

printf("format", variable);

scanf("format", &variable);

Examples

```
int a = 10;
printf("%d", a);
```

%d → prints integer value

```
float b = 3.14;
printf("%f", b);
```

%f → prints floating value

```
char ch = 'A';
printf("%c", ch);
```

%c → prints character

Format Specifiers

Format Specifier	Data Type	Example
%c	Character	'A'
%d	Signed Integer	10
%i	Integer	10
%u	Unsigned Integer	100
%hd	Short Integer	25
%hu	Unsigned Short Integer	25
%ld	Long Integer	100000
%lu	Unsigned Long Integer	100000
%lld	Long Long Integer	1234567890

%llu	Unsigned Long Long Integer	1234567890
%f	Float	3.14
%lf	Double	3.14159
%Lf	Long Double	3.1415926535
%e	Scientific Notation	1.23e+03
%E	Scientific Notation	1.23E+03
%g	Shortest Representation of Float/Double	3.14
%G	Shortest Representation of Float/Double	3.14
%o	Octal Integer	012
%x	Hexadecimal (lowercase)	0x1A
%X	Hexadecimal (uppercase)	0X1A
%p	Address (Pointer)	0x7ffe1234

%s	String	"Hello"
%%	Percent Sign	%

Example Program

```c
#include <stdio.h>
int main()
{
  int a = 10;
  float b = 5.5;
  char ch = 'A';
  printf("a = %d\n", a);
  printf("b = %f\n", b);
  printf("ch = %c\n", ch);
  return 0;
}
```

Important Points

- Always starts with %
- Must match data type of variable
- Used in input and output functions
- Wrong specifier → incorrect output

Variable in C

Definition

A **variable** is a named memory location used to store data that can be changed during program execution.

Why Do We Need Variables?

Variables are used to:

- Store data in memory.

- Perform calculations.
- Store user input.
- Store intermediate results.
- Manipulate and process information.

Example

int age = 25;

Here:

- int → Data type
- age → Variable name
- 25 → Value stored in memory

Syntax
Variable Declaration

data_type variable_name;

Examples

int age;

float salary;

char grade;

Variable Initialization

Assigning an initial value to a variable is called **initialization**.

Example:

int age = 25;

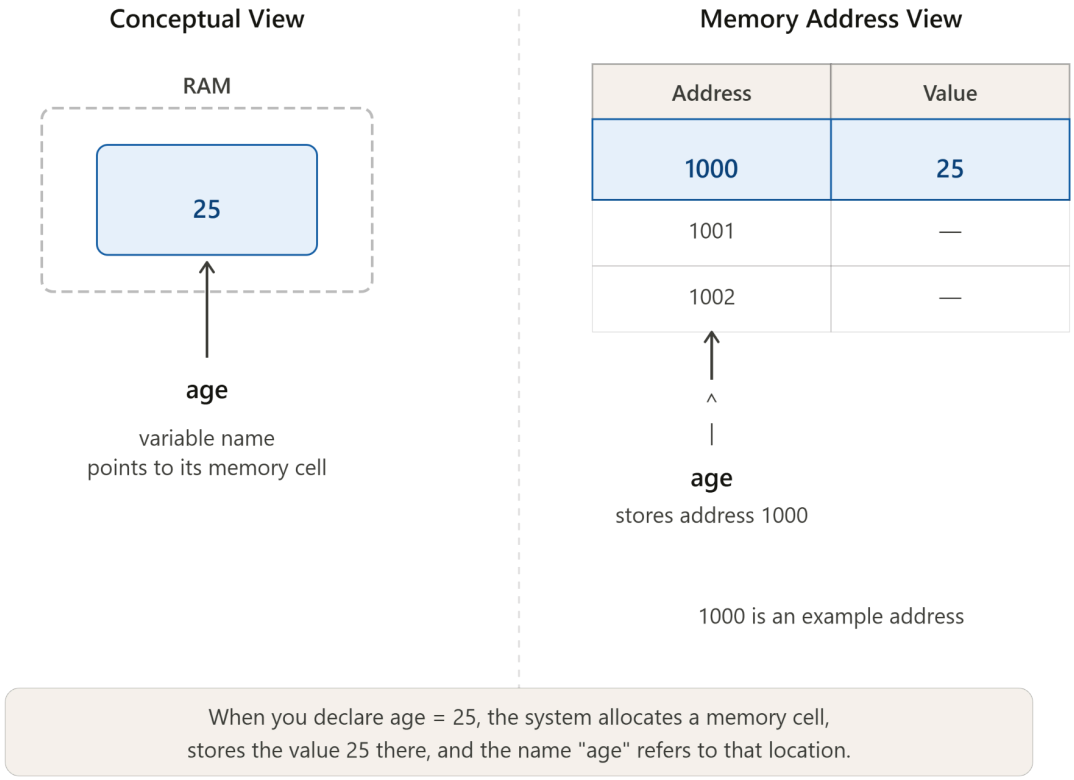
float pi = 3.14;

char grade = 'A';

Memory Representation

Example:

int age = 25;



How a Variable Works?

int x = 10;

Memory: x = 10

After: x = 50;

Memory: x = 50

The old value (10) is replaced by the new value (50).

Rules for Naming Variables

1. Must Begin with a Letter or Underscore

Valid	Invalid
int _count;	int 1age;

2. Cannot Contain Spaces

Valid	Invalid
int total_marks;	int total marks;

3. Cannot Use Keywords

Valid : int number;

Invalid : int float; → float is a keyword.

4. Can Contain Digits

Valid :

int student1;

int num123;

5. Only Letters, Digits and Underscore Allowed

Valid

int total_marks;

Invalid

int total-marks;

int total@marks;

6. Case Sensitive

int age;

int Age;

These are two different variables.

Types of Variables

Type	Example
Integer	int age = 25;
Character	char grade = 'A';
Float	float pi = 3.14;
Double	double salary = 25000.50;

Example Program

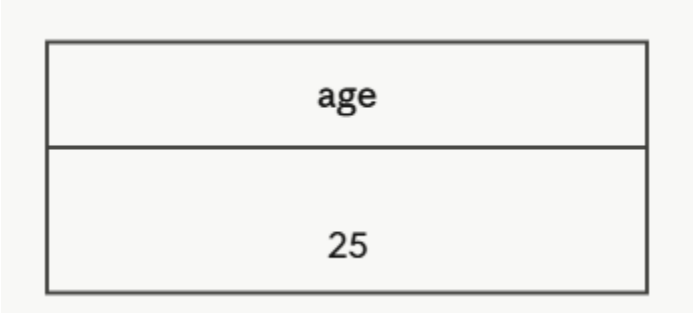
```
#include <stdio.h>
int main()
{
   int age = 25;
   printf("Age = %d", age);
   return 0;
}
```

Output

Age = 25

Real-Life Analogy

Think of a variable as a **labeled box**.



- Box Name → Variable Name (age)
- Content → Variable Value (25)
- Box Size → Data Type (int)

Characteristics of a Variable

1. Has a unique name.
2. Occupies memory.
3. Stores a value.
4. Value can change during execution.
5. Must be declared before use.
6. Has a specific data type.

First C Program – Hello World

The first program that beginners learn in C is the **Hello World Program**.

```
#include <stdio.h>

int main()
{
   printf("Hello World");
```

```
    return 0;
}
```

Output

Hello World

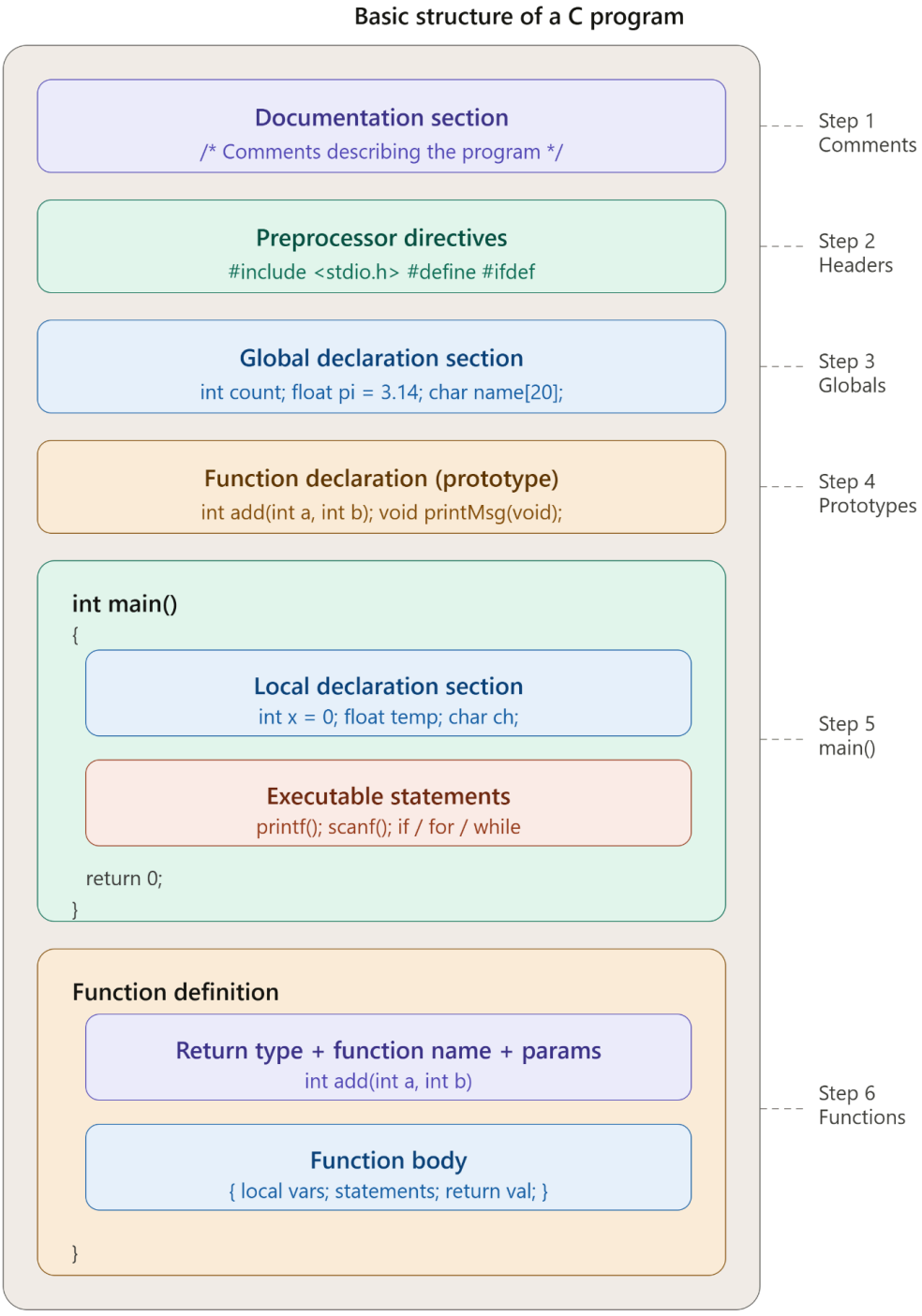
What is a Program?

A **program** is a set of instructions written in a programming language to perform a specific task.

Examples

- Calculator Program
- LED Blinking Program
- ATM Software
- Temperature Monitoring System
- Student Management System

Basic Structure of a C Program



Explanation

1. Documentation Section

/* Program to Add Two Numbers */
/* Author: ABC */

Purpose

- Describes the program.
- Contains comments for the programmer.

Features

- Ignored by the compiler.
- Used for readability and maintenance.

Example
/* Program to display Hello World */

2. Preprocessor Directive Section

```
#include <stdio.h>
#define PI 3.14
```

Purpose : Executed before compilation begins.

Common Directives

Directive	Purpose
#include	Include header file
#define	Define macro
#ifdef	Conditional compilation
#ifndef	Conditional compilation

Example
#include <stdio.h>

Includes declarations of:

printf()
scanf()

3. Global Declaration Section

```
int count;
float pi = 3.14;
char name[20];
```

Purpose : Declares global variables.

Characteristics

- Declared outside all functions.
- Accessible by all functions.
- Exists throughout program execution.

Example
int total;

4. Function Declaration (Prototype)

```
int add(int a, int b);
void printMsg(void);
```

Purpose :

Tells the compiler:

- Function name
- Return type
- Number of arguments
- Type of arguments

Example

```
int add(int, int);
```

Meaning:

Function Name : add
Return Type : int
Parameters : int, int

5. main() Function

int main()

```
{
  ...
}
```

Purpose

main() is the entry point of a C program. Execution of every C program starts from the main() function. When a program is executed, the operating system first calls main(), and the statements inside main() are executed sequentially.

When a program runs:

Operating System → main()

The operating system automatically calls main().

Inside main()
A) Local Declaration Section

```
int x = 0;
float temp;
char ch;
```

Purpose :

Declares local variables.

Characteristics

- Accessible only within main().
- Created when function starts.
- Destroyed when function ends.

B) Executable Statements

These are actual instructions executed by the CPU.

Examples

```
printf("Hello");
scanf("%d",&num);

if(num>0)
{
}
```

```
for(i=0;i<10;i++)
{
}
```

Types

- Input Statements : scanf()
- Output Statements : printf()
- Decision Statements
  if
  switch

- Loop Statements
  for
  while
  do-while

C) Return Statement

What is return in C?

The **return statement** is used to:

1. **Terminate (end) a function.**
2. **Return a value to the calling function.**

Syntax : return value;

Example:

```
return 10;
```

The function ends and sends the value 10 back to the caller.
What is return 0 in main()?

```
int main()
{
  return 0;
}
```

Here:

- main() is called by the **Operating System (OS)**.
- When main() finishes, it returns a value to the OS.
- This value is called the **Exit Status** or **Return Code**.



Meaning of return 0, return 1, and return -1

Return Value	Meaning
return 0;	Program executed successfully
return 1;	General error/failure
return -1;	Program-defined error/failure
return 2;	Another error code (program-defined)
return n;	Any non-zero value usually indicates an error

Examples

Successful Execution

```c
#include <stdio.h>

int main()
{
   printf("Program Executed Successfully");
   return 0;
}
```

Meaning:

Exit Status = 0
Success

Error Condition

```c
#include <stdio.h>

int main()
{
   printf("File Not Found");
   return 1;
}
```

Meaning:

Exit Status = 1
Error

Custom Error

```c
#include <stdio.h>

int main()
{
   printf("Invalid Input");
   return -1;
}
```

Meaning:

Exit Status = -1, Custom Error

Common Return Codes

Return Statement	Returned To	Meaning
return 0;	Operating System	Successful execution
return 1;	Operating System	General error
return -1;	Operating System	Custom/program-defined error
return 2;	Operating System	Another custom error
return value;	Calling function/OS	Returns specified value

6. Function Definition Section

Functions are usually written after main().

Example:

```
int add(int a, int b)
{
   return a + b;
}
```

Parts of Function Definition
Function Header:

int add(int a, int b)

Part	Meaning
int	Return type
add	Function name
int a, int b	Parameters

Function Body

```
{
  return a + b;
}
```

Contains:

- Local variables
- Statements
- Return value

Complete Example

```c
/* Program to Add Two Numbers */

#include <stdio.h>

int add(int,int);

int main()
{
  int a = 10;
  int b = 20;
  int sum;

  sum = add(a,b);

  printf("Sum = %d",sum);

  return 0;
}

int add(int x,int y)
{
  return x + y;
}
```

Detailed Analysis of Hello World Program

```c
#include <stdio.h>
```

```c
int main()
{
   printf("Hello World");

   return 0;
}
```

Step 1

#include <stdio.h>

Includes Standard Input Output library.

Step 2

int main()

Program execution starts here.

Step 3

{

Beginning of function body.

Step 4

printf("Hello World");

Prints:

Hello World

on the screen.

Step 5

return 0;

Returns success status to the operating system.

Step 6

}

End of function body.

Types of Instructions in C

A C program consists of different types of instructions.

1. Type Declaration Instructions

Used to declare variables.

Example:

```
 int a;

float b;

char c;
```

2. Arithmetic Instructions

Used for calculations.

Example:

```
sum = a + b;

avg = total / count;
```

Operators:

+, - , *, / ,%

3. Control Instructions

Used to control program flow.

Examples:

- **Decision Making**
  - If
  - If else

    - Switch
- **Looping**
    - for()
    - while()
    - do while()
- **Branching**
    - Break
    - continue
    - goto

4. Input/Output Instructions

Used to take input and display output.

Input

scanf("%d",&num);

Output

printf("%d",num);

printf() and scanf() in C

Both printf() and scanf() are standard library functions declared in: #include <stdio.h>

They are used for **Input and Output (I/O)** operations.

1. printf() Function

printf() is a library function used to **display output on the screen**.

Syntax :

printf("format string", argument1, argument2, ...);

Example :

int age = 25;

printf("Age = %d", age);

Output

Age = 25

Arguments of printf()

printf("Age = %d", age);

Argument	Purpose
"Age = %d"	Format string
age	Value to be printed

Format String

"Age = %d"

Contains:

- Normal text → Age =
- Format specifier → %d

Actual Value

age : The value of age replaces %d.

Return Value of printf()

printf() returns the **number of characters printed**.

Example:

```
int n;
n = printf("Hello");
```

Output:

Hello

Value of n: 5 → because 5 characters were printed.

2. scanf() Function

scanf() is a library function used to **read input from the keyboard**.

Syntax

scanf("format string", &variable);

Example

int age;

scanf("%d", &age);

If user enters: 25

Then: age = 25

Arguments of scanf()

scanf("%d", &age);

Argument	Purpose
%d	Input format
&age	Address where value is stored

Why &?

&age

means:

Address of variable age, scanf() stores the entered value at this memory location.

Return Value of scanf()

scanf() returns the **number of input items successfully read and assigned**.

Example:

```
int age;
int n = scanf("%d",&age);
```

Input: 25

Value of n is 1 because one item was successfully read.

Example Program

```
#include <stdio.h>
int main()
{
   int age;
   printf("Enter Age: ");
   scanf("%d", &age);
   printf("Age = %d", age);
   return 0;
}
```

Execution

Enter Age: 25

Age = 25

Comparison Between printf() and scanf()

Feature	printf()	scanf()
Purpose	Output Function	Input Function

Header File	stdio.h	stdio.h
Reads/Writes	Writes to Screen	Reads from Keyboard
Syntax	printf("format",value);	scanf("format",&variable);
Return Value	Number of characters printed	Number of items successfully read

Operators in C

An **operator** is a special symbol that performs a specific operation on one or more operands to produce a result.

Example

a + b

Where:

- a and b → Operands
- + → Operator

What is an Operand?

The data on which an operator performs an operation is called an **operand**.

Example:

10 + 20

- 10 and 20 → Operands
- + → Operator

Classification of Operators

- Based on Number of Operands
  - Unary Operators
  - Binary Operators
  - Ternary Operators
- Based on Functionality
  - Arithmetic Operators
  - Relational Operators
  - Logical Operators
  - Assignment Operators
  - Increment and Decrement Operators
  - Bitwise Operators
  - Conditional (Ternary) Operator
  - sizeof Operator
  - Comma Operator
  - Type Cast Operator
  - Special Operators
    - Address Operator (&)
    - Pointer Operator (*)
    - Member Access Operator (.)
    - Arrow Operator (->)

Based on Number of Operands

1. Unary Operator

Works on **one operand**.

Examples :

++a , --a, a++, a– , sizeof(a)

2. Binary Operator

Works on **two operands**.

Examples:

a + b, a - b, a * b, a / b, a > b, a && b

3. Ternary Operator

Works on **three operands**.

Example:

condition ? expression1 : expression2

a>b ? a : b

Based on Functionality

1. Arithmetic Operators

Used for mathematical calculations.

Operator	Meaning
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulus

Example

```
#include <stdio.h>

int main()
{
   int a=10,b=3;
```

```
    printf("%d\n",a+b);
    printf("%d\n",a-b);
    printf("%d\n",a*b);
    printf("%d\n",a/b);
    printf("%d\n",a%b);

    return 0;
}
```

Output

13

7

30

3

1

2. Relational Operators

Relational operators are used to **compare two values or expressions**.

The result of a relational operation is always either:

- **1 → True**
- **0 → False**

Relational Operators in C

Operator	Meaning	Example	Result
>	Greater than	10 > 5	1

<	Less than	10 < 5	0
>=	Greater than or equal to	10 >= 10	1
<=	Less than or equal to	5 <= 10	1
==	Equal to	10 == 10	1
!=	Not equal to	10 != 5	1

Example :

```
#include <stdio.h>

int main()
{
  int a, b;

  printf("Enter value of a: ");
  scanf("%d", &a);

  printf("Enter value of b: ");
  scanf("%d", &b);

  printf("\nValues:\n");
  printf("a = %d\n", a);
  printf("b = %d\n", b);

  printf("\nRelational Operator Results:\n");

  printf("a < b  = %d\n", a < b);
```

```
    printf("a > b  = %d\n", a > b);
    printf("a <= b = %d\n", a <= b);
    printf("a >= b = %d\n", a >= b);
    printf("a == b = %d\n", a == b);
    printf("a != b = %d\n", a != b);

    return 0;
}
```

Input:

Enter value of a: 10

Enter value of b: 20

Output:

a < b = 1

a > b = 0

a <= b = 1

a >= b = 0

a == b = 0

a != b = 1

Input:

Enter value of a: 15

Enter value of b: 15

Output:

a < b = 0

a > b = 0

a <= b = 1

a >= b = 1

a == b = 1

a != b = 0

3. Logical Operators

Logical operators are used to **combine two or more conditions** and produce a single result.

The result of a logical operation is always:

- **1** → True
- **0** → False

Logical Operators Table

Operator	Name	Description	Example
&&	Logical AND	Returns True (1) only if **both conditions are true**	(a > b && b > c)
||	Logical OR	Returns false (0) only if **both conditions are false**	(a > b || b > c)
!	Logical NOT	Reverses the result of a condition	!(a > b)

1. Logical AND (&&)

The Logical AND operator combines two conditions. The result is **True (1)** only when **both conditions are true**.

Syntax: condition1 && condition2

Example : (10 > 5 && 20 > 10)
Evaluation: (True && True)

Result: 1

AND Truth Table

Condition 1	Condition 2	Result
0	0	0

0	1	0
1	0	0
1	1	1

Both conditions must be true.

Example
int age = 25;
int salary = 50000;
(age >= 18 && salary > 30000)

Result: (1 && 1)
Output: 1

2. Logical OR (||)

The Logical OR operator combines two conditions. The result is **True (1)** if **at least one condition is true**.

Syntax : condition1 || condition2

Example : (10 > 5 || 20 < 10)

Evaluation: (True || False)

Result: 1

OR Truth Table

Condition 1	Condition 2	Result
0	0	0
0	1	1
1	0	1
1	1	1

Only one condition needs to be true.

Example
int marks = 45;

int sports = 1;

(marks >= 50 || sports == 1)

Evaluation: (0 || 1)

Output: 1

3. Logical NOT (!)

The Logical NOT operator reverses the result of a condition.

- True becomes False
- False becomes True

Syntax : !condition

Example : !(10 > 5)

Evaluation: !True

Result: 0

NOT Truth Table

Condition	Result after NOT
0	1
1	0

Example
int a = 10;
int b = 20;

!(a > b)

Evaluation:

!(10 > 20) → !(0) → 1

Output: 1

Example Program

```
#include <stdio.h>
```

```c
int main()
{
   int a = 10;
   int b = 20;

   printf("AND = %d\n", (a < b && b > 0));
   printf("OR  = %d\n", (a > b || b > 0));
   printf("NOT = %d\n", !(a > b));

   return 0;
}
```

Output :
AND = 1
OR = 1
NOT = 1

4. Assignment Operators

An **Assignment Operator** is used to assign a value to a variable.The value on the **right-hand side (RHS)** is assigned to the variable on the **left-hand side (LHS)**.

Types of Assignment Operators

Operator	Meaning	Example	Equivalent To
=	Assign	a = 10	a = 10
+=	Add and Assign	a += 5	a = a + 5
-=	Subtract and Assign	a -= 5	a = a - 5
*=	Multiply and Assign	a *= 5	a = a * 5

/=	Divide and Assign	a /= 5	a = a / 5
%=	Modulus and Assign	a %= 5	a = a % 5

Example

int a=10;

a+=5;

Equivalent to: a=a+5;

Program

```
#include <stdio.h>
int main()
{
   int a;

   /* Simple Assignment */
   a = 10;
   printf("a = %d\n", a);

   /* Add and Assign */
   a += 5;
   printf("a += 5 : %d\n", a);

   /* Subtract and Assign */
   a -= 3;
   printf("a -= 3 : %d\n", a);

   /* Multiply and Assign */
   a *= 2;
   printf("a *= 2 : %d\n", a);

   /* Divide and Assign */
```

```c
    a /= 4;
    printf("a /= 4 : %d\n", a);

    /* Modulus and Assign */
    a %= 3;
    printf("a %%= 3 : %d\n", a);

    return 0;
}
```

Output:
a = 10

a += 5 : 15

a -= 3 : 12

a *= 2 : 24

a /= 4 : 6

a %= 3 : 0

6. Bitwise Operator

Before learning bitwise operators, you must understand **how numbers are stored in memory**, especially **negative numbers**.

Step 1: How Positive Numbers are Stored

Consider: int a = 5;

Binary representation of 5: 00000101

(assuming 8-bit representation for understanding)

Bit Position	**7**	**6**	**5**	**4**	**3**	**2**	**1**	**0**

Binary	0	0	0	0	0	1	0	1

Decimal value:

= 0×128 + 0×64 + 0×32 + 0×16+ 0×8 + 1×4 + 0×2 + 1×1
= 5

Step 2: How Negative Numbers are Stored

Modern computers use **2's Complement Representation**.

Example: Store -5

Step 1: Write +5 in Binary
+5 = 00000101

Step 2: Take 1's Complement

Invert every bit.

00000101 —-> 11111010

Step 3: Add 1
11111010
+ 1

11111011

Therefore:

-5 = 11111011

Verification

Convert back: 11111011

Take 1's complement: 00000100

Add 1: 00000101 = 5

q2Hence: 11111011 = -5

Why Use 2's Complement?

Advantages:

1. Only one representation for zero.
2. Easier arithmetic operations.
3. Faster hardware implementation.

Range of Signed 8-bit Numbers:

Binary	Decimal
00000000	0
00000001	1
...	...
01111111	+127
10000000	-128
...	...
11111111	-1

Bitwise Operators

Bitwise operators work directly on bits.

Operator	Name
&	Bitwise AND
|	Bitwise OR
^	Bitwise XOR
~	Bitwise NOT
<<	Left Shift
>>	Right Shift

1. Bitwise AND (&)

Rule

1 & 1 = 1
1 & 0 = 0
0 & 1 = 0
0 & 0 = 0

Example : 5 & 3

Binary:

```
5  =  0 1 0 1
3  =  0 0 1 1
-----------------
       0 0 0 1
```

Result:1

Program

```c
#include <stdio.h>

int main()
{
   int a = 5;
   int b = 3;

   printf("%d", a & b);

   return 0;
}
```

Output: 1

2. Bitwise OR (|)

Rule

1 | 1 = 1
1 | 0 = 1
0 | 1 = 1
0 | 0 = 0

Example:

5 = 0 1 0 1
3 = 0 0 1 1
---
0 1 1 1

Result: 7

3. Bitwise XOR (^)

1 ^ 1 = 0
1 ^ 0 = 1
0 ^ 1 = 1
0 ^ 0 = 0

Example:

5 = 0 1 0 1
3 = 0 0 1 1
---
0 1 1 0

Result: 6

4. Bitwise NOT (~)

Inverts every bit.

Example: ~5

Binary:

5 = 00000101
~5 = 11111010

This is a negative number.

Convert back: 11111010

1's complement: 00000101

Add 1: 00000110 = 6

Therefore: ~5 = -6

Left Shift (<<)
Definition : Shifts bits to the left.

Syntax: value << n

Means: Shift bits left by n positions
Example : 5 << 1

Binary: 00000101

Shift left: 00001010

Result: 10

Formula

For positive numbers: $x << n = x \times 2^n$

Example:

5 << 2

= 5 × 4

= 20

Verification:

00000101

Shift 2:

00010100

= 20

Right Shift (>>)
Definition : Shifts bits to the right.

Syntax: value >> n

Example : 20 >> 2

Binary: 00010100

Shift right: 00000101

Result: 5

Formula

For positive numbers: $x >> n = x / 2^n$

Example:

20 >> 2

= 20 / 4

= 5

Left Shift of Negative Number

Consider: -5 << 1

Binary:

-5 = 11111011

Shift left:

11110110

This gives:

-10

Mathematically:

-5 × 2

= -10

Important

For signed negative numbers: -5 << 1

is **implementation-dependent/undefined behavior in C** if overflow or sign changes occur.

For exams, you'll often see:

-5 << 1 = -10

but in real C programming, avoid left-shifting negative signed integers.

Right Shift of Negative Number

Consider: -8 >> 1

Binary: -8 = 11111000

Most systems perform **Arithmetic Right Shift**.

Shift:

11111100

Notice:

The leftmost bit (sign bit) remains 1.

Result: -4

Example : -8 >> 1 = -4

Why Sign Bit is Preserved?

Negative numbers use 2's complement.

If sign bit were replaced with 0: 01111100

it would become positive,therefore most processors copy the sign bit.

This is called:

Arithmetic Right Shift
11111000 → 11111100

Program Demonstrating Shift Operators

```
#include <stdio.h>

int main()
{
   int a = 5;
```

```
    int b = -8;

    printf("5 << 1 = %d\n", a << 1);
    printf("5 << 2 = %d\n", a << 2);

    printf("20 >> 1 = %d\n", 20 >> 1);
    printf("20 >> 2 = %d\n", 20 >> 2);

    printf("-8 >> 1 = %d\n", b >> 1);
    printf("-8 >> 2 = %d\n", b >> 2);

    return 0;
}
```

Output (on most systems)

5 << 1 = 10
5 << 2 = 20

20 >> 1 = 10
20 >> 2 = 5

-8 >> 1 = -4
-8 >> 2 = -2

Difference Between Logical and Arithmetic Right Shift?

Logical Shift	Arithmetic Shift
Fills with 0	Fills with sign bit
Used for unsigned numbers	Used for signed numbers
Sign may change	Sign preserved

Summary Table

Operator	Meaning	Example	Result
&	AND	5 & 3	1
|	OR	5 | 3	7
^	XOR	5 ^ 3	6

~	NOT	~5	-6
<<	Left Shift	5<<1	10
>>	Right Shift	20>>2	5

7. Increment and Decrement Operators

Increment (++) and Decrement (--) operators are **unary operators** because they operate on only one operand (variable).

They are used to:

- Increase a variable value by 1
- Decrease a variable value by 1

Increment Operator (++)

The increment operator increases the value of a variable by 1.

Syntax
Variable++; or ++variable;

Example

```
int a = 10;

a++;
```

After execution: a = 11

Equivalent to: a = a + 1;

Decrement Operator (--)

The decrement operator decreases the value of a variable by 1.

Syntax:

Variable--; Or --variable;

Example:

```
int a = 10;

a--;
```

After execution: a = 9

Equivalent to: a = a - 1;

Types of Increment Operators

There are two types:

1. Pre-Increment (++a)
2. Post-Increment (a++)

1. Pre-Increment (++a)

In Pre-Increment:

- First increment the value.
- Then use the value.

Syntax : ++variable;
Example

```
int a = 10;

int b = ++a;
```

Step 1 : Increment first: a = 11;

Step 2 : Assign value to b: b = 11;

Final Values
a = 11
b = 11

Program for Pre-Increment

```
#include <stdio.h>

int main()
{
```

```
    int a = 10;
    int b;

    b = ++a;

    printf("a = %d\n", a);
    printf("b = %d\n", b);

    return 0;
}
```

Output

a = 11
b = 11

2. Post-Increment (a++)

In Post-Increment:

- First use the current value.
- Then increment the value.

Syntax : variable++;

Example

```
int a = 10;

int b = a++;
```

Step 1 : Assign current value: b = 10;

Step 2 : Increment a: a = 11;

Final Values
a = 11
b = 10

Program for Post-Increment

```
#include <stdio.h>

int main()
{
```

```
    int a = 10;
    int b;

    b = a++;

    printf("a = %d\n", a);
    printf("b = %d\n", b);

    return 0;
}
```

Output

a = 11
b = 10

Comparison of Pre and Post Increment

Feature	Pre-Increment (++a)	Post-Increment (a++)
Increment occurs	Before use	After use
Value assigned	Incremented value	Original value
Example	b = ++a	b = a++
Result if a=10	a=11, b=11	a=11, b=10

Types of Decrement Operators

There are two types:

1. Pre-Decrement (--a)
2. Post-Decrement (a--)

3. Pre-Decrement (--a)

- First decrease the value.
- Then use the value.

Example

```
int a = 10;

int b = --a;
```

Step 1: Decrement first:
a = 9;

Step 2: Assign: b = 9;

Final Values
a = 9
b = 9
Program

```
#include <stdio.h>

int main()
{
   int a = 10;
   int b;

   b = --a;

   printf("a = %d\n", a);
   printf("b = %d\n", b);

   return 0;
}
```

Output
a = 9
b = 9

4. Post-Decrement (a--)

- First use the value.
- Then decrease the value.

Example

int a = 10;

int b = a--;

Step 1 : Assign current value: b = 10;

Step 2 : Decrement: a = 9;

Final Values
a = 9
b = 10

Program

```
#include <stdio.h>

int main()
{
   int a = 10;
   int b;

   b = a--;

   printf("a = %d\n", a);
   printf("b = %d\n", b);

   return 0;
}
```

Output

a = 9
b = 10

One Program Demonstrating All Four Cases

```
#include <stdio.h>

int main()
{
   int a, b;

   a = 10;
   b = ++a;
   printf("Pre Increment : a=%d b=%d\n", a, b);

   a = 10;
   b = a++;
```

```
    printf("Post Increment: a=%d b=%d\n", a, b);

    a = 10;
    b = --a;
    printf("Pre Decrement : a=%d b=%d\n", a, b);

    a = 10;
    b = a--;
    printf("Post Decrement: a=%d b=%d\n", a, b);

    return 0;
}
```

Output

Pre Increment : a=11 b=11
Post Increment: a=11 b=10

Pre Decrement : a=9 b=9
Post Decrement: a=9 b=10

7. Conditional (Ternary) Operator

The **Conditional Operator (?:)** is the only **ternary operator** in C.

It works as a shorthand form of the if-else statement.

Since it requires **three operands**, it is called a **Ternary Operator**.

Syntax

condition ? expression1 : expression2;

Components

Part	Meaning
condition	Expression to be tested

?	Separates condition from true part
expression1	Executed if condition is true
:	Separates true and false parts
expression2	Executed if condition is false

Working :

The conditional operator evaluates the condition.

If condition is TRUE

condition ? expression1 : expression2

expression1 is executed.

If condition is FALSE

condition ? expression1 : expression2

expression2 is executed.

Example 1: Find Larger Number

```
#include <stdio.h>
int main()
{
   int a = 10;
   int b = 20;

   int max;
```

```
    max = (a > b) ? a : b;

    printf("Maximum = %d", max);

    return 0;
}
```

Output

Maximum = 20

Step-by-Step Execution

Statement

max = (a > b) ? a : b;

Given:

a = 10 , b = 20

Step 1:

Evaluate condition:

a > b → 10 > 20

Result: 0 (False)

Step 2:

Since condition is false, b is selected.

Step 3: max = 20;

Equivalent if-else

The above statement is equivalent to:

```
if(a > b)
```

```
{
   max = a;
}
else
{
   max = b;
}
```

Example 2: Check Even or Odd

```
#include <stdio.h>

int main()
{
  int num;

  printf("Enter number: ");
  scanf("%d", &num);

  (num % 2 == 0) ?
  printf("Even Number") :
  printf("Odd Number");

  return 0;
}
```

Input : 10

Output : Even Number

Input : 15

Output : Odd Number

Example 3: Find Greatest of Two Numbers

```
#include <stdio.h>
```

```c
int main()
{
   int a,b;

   printf("Enter two numbers: ");
   scanf("%d%d",&a,&b);

   printf("Greater Number = %d",
          (a>b)?a:b);

   return 0;
}
```

Example 4: Positive or Negative Number

```c
#include <stdio.h>

int main()
{
   int num;

   printf("Enter number: ");
   scanf("%d",&num);

   (num>=0)? printf("Positive Number") :  printf("Negative Number");

   return 0;
}
```

Nested Conditional Operator:

A conditional operator can be used inside another conditional operator.

Example: Largest of Three NumbersExample

```c
#include <stdio.h>

int main()
```

```c
{
   int a,b,c,max;

   printf("Enter three numbers: ");
   scanf("%d%d%d",&a,&b,&c);

   max = (a>b) ?  ((a>c)?a:c)  :  ((b>c)?b:c);

   printf("Largest Number = %d", max);

   return 0;
}
```

Input: 10 50 30

Output: Largest Number = 50

Advantages of Conditional Operator

1. Shorter than if-else.
2. Improves readability for simple conditions.
3. Useful in assignments.
4. Reduces number of lines of code.

Example:

if-else

```
if(a>b)

{

  max=a;

}

else

{

  max=b;
```

}

Conditional Operator

Max = (a>b)?a:b;

Limitations

- Not suitable for complex logic.
- Excessive nesting makes code difficult to read.

Use if-else for such cases.

Difference Between if-else and Conditional Operator

if-else	Conditional Operator
Statement	Operator
Multiple statements possible	Single expression
Suitable for complex logic	Suitable for simple decisions
More lines of code	Compact code

8. sizeof Operator

Returns memory occupied by a variable or datatype.

Syntax : sizeof(variable)

sizeof(datatype)

Example

printf("%d",sizeof(int));

Output : 4 (May be 2 or 4 depending on compiler)

sizeof() is a **special operator** in C used to determine the **memory size (in bytes)** occupied by:

- A data type
- A variable
- An array
- A structure
- A union

The value returned by sizeof() is the number of bytes required to store the operand.

Syntax
1. Size of a Data Type

sizeof(data_type)

Example:

sizeof(int)

sizeof(char)

sizeof(float)

2. Size of a Variable

sizeof(variable_name)

Example:
int a;
sizeof(a)

Return Type

The sizeof() operator returns an unsigned integer value of type, **size_t** which represents the size in bytes.

Program 1: Size of Basic Data Types

```c
#include <stdio.h>

int main()
{
   printf("Size of char   = %u bytes\n", sizeof(char));
   printf("Size of int    = %u bytes\n", sizeof(int));
   printf("Size of float  = %u bytes\n", sizeof(float));
   printf("Size of double = %u bytes\n", sizeof(double));

   return 0;
}
```

Possible Output

Size of char   = 1 bytes

Size of int   = 4 bytes

Size of float = 4 bytes

Size of double = 8 bytes

Note: The output may vary depending on the compiler and processor architecture.

Program 2: Size of Variables

```c
#include <stdio.h>

int main()
{
   char c;
   int i;
   float f;
   double d;

   printf("Size of c = %u bytes\n", sizeof(c));
   printf("Size of i = %u bytes\n", sizeof(i));
   printf("Size of f = %u bytes\n", sizeof(f));
   printf("Size of d = %u bytes\n", sizeof(d));
```

```
    return 0;
}
```

Using sizeof() with Arrays
Example : int arr[10];

Each integer occupies 4 bytes.

Total size: 10 × 4 = 40 bytes

Program

```
#include <stdio.h>

int main()
{
    int arr[10];

    printf("Size of array = %u bytes\n", sizeof(arr));

    return 0;
}
```

Output : Size of array = 40 bytes

Finding Number of Elements in an Array

This is one of the most important uses of sizeof().

```
#include <stdio.h>

int main()
{
    int arr[10];

    int elements;

    elements = sizeof(arr) / sizeof(arr[0]);
```

```
    printf("Number of elements = %d\n", elements);

    return 0;
}
```

Calculation : sizeof(arr)

Returns: 40 , because 10 × 4 = 40, and sizeof(arr[0]) , returns 4

Therefore: 40 / 4 = 10

Using sizeof() with Expressions

```
#include <stdio.h>

int main()
{
    printf("%u\n", sizeof(5));
    printf("%u\n", sizeof(5.5));
    printf("%u\n", sizeof('A'));

    return 0;
}
```

Typical Output

4

8

4

Using sizeof() with Structures

```
#include <stdio.h>

struct Student
{
    int roll;
```

```
    char grade;
};

int main()
{
    struct Student s;

    printf("Size = %u bytes\n", sizeof(s));

    return 0;
}
```

Output depends on compiler padding and alignment.

Important Point: sizeof() is Evaluated at Compile Time

Example:

int a = 10;

printf("%u", sizeof(a));

The compiler already knows the size of a.

No runtime calculation is required.

Difference Between sizeof() and strlen()

sizeof()	**strlen()**
Operator	Function
Returns memory occupied	Returns string length

Works on any data type	Works only on strings
Compile-time evaluation	Runtime evaluation
Counts bytes	Counts characters

Example

char str[] = "HELLO";

sizeof(str)

Returns:6 → (5 characters + '\0')

strlen(str) → returns: 5

9. Comma Operator

Evaluates expressions from left to right.

Example

int a,b;

a=(b=5,b+10);

Evaluation: b=5, b+10

Result: a = 15

11. Address Operator (&)

Returns memory address.

Example

int a=10;

printf("%p",&a);

Output : 1000 (example)

12. Pointer Operator (*)

Used to access value stored at an address.

Example

int a=10;

int *p=&a;

printf("%d",*p);

Output : 10

Operator Precedence and Associativity

Operator Precedence

Operator precedence determines **which operator is evaluated first** when an expression contains multiple operators.

Example:

int x = 10 + 5 * 2;

`*` has higher precedence than `+`, so: 10 + (5 × 2) = 20

Operator Associativity

Associativity determines the **direction in which operators having the same precedence are evaluated**.

- **Left to Right:** `+`, `-`, `*`, `/`
- **Right to Left:** assignment operators such as `=`, `+=`, `-=`

Example:

int x = 20 - 10 + 5;

Both - and + have the same precedence, so evaluation is **left to right**:

(20 - 10) + 5 = 15

Operator	*Description*	*Associativity*
() **[]** **.** **->** **++ --**	Parentheses or function call Brackets or array subscript Dot or Member selection operator Arrow operator Postfix increment/decrement	left to right
++ -- **+ -** **! ~** **(type)** ***** **&** **sizeof**	Prefix increment/decrement Unary plus and minus not operator and bitwise complement type cast Indirection or dereference operator Address of operator Determine size in bytes	right to left
*** / %**	Multiplication, division and modulus	left to right
+ -	Addition and subtraction	left to right
<< >>	Bitwise left shift and right shift	left to right
< <= **> >=**	relational less than/less than equal to relational greater than/greater than or equal to	left to right
== !=	Relational equal to or not equal to	left to right
&&	Bitwise AND	left to right
^	Bitwise exclusive OR	left to right
|	Bitwise inclusive OR	left to right
&&	Logical AND	left to right
||	Logical OR	left to right
? :	Ternary operator	right to left
= **+= -=** ***= /=** **%= &=** **^= |=** **<<= >>=**	Assignment operator Addition/subtraction assignment Multiplication/division assignment Modulus and bitwise assignment Bitwise exclusive/inclusive OR assignment	right to left
,	comma operator	left to right

Practice Question:

1. **A token is:**
   A) Line of code
   B) Smallest unit of program
   C) Function
   D) Loop

2. **Which of the following is NOT a token?**
   A) Keyword
   B) Identifier
   C) Constant
   D) Comment

3. **How many types of tokens are there in C?**
   A) 2
   B) 3
   C) 5
   D) 7

4. **Which of the following is a token?**
   A) int
   B) main()
   C) #include
   D) All of the above

5. **Which token is used to perform operations?**
   A) Identifier
   B) Constant
   C) Operator
   D) Keyword

6. **Identifier is:**
   A) Reserved word
   B) Name of variable/function
   C) Constant value
   D) Operator

7. Which of the following is a valid identifier?

A) 2value
B) total_marks
C) float
D) @data

8. Which is an invalid identifier?

A) sum1
B) _count
C) int
D) marks

9. Identifier cannot start with:

A) Letter
B) Underscore
C) Number
D) Alphabet

10. Which of the following rules is correct for identifiers?

A) Can contain spaces
B) Can start with number
C) Cannot be keyword
D) Can contain special symbols

11. Data type defines:

A) Variable name
B) Type of data stored
C) Only value
D) Only memory

12. Which of the following is a primary data type?

A) Array
B) Pointer
C) int
D) Structure

13. Which of the following is NOT a primary data type?

A) int

B) char
C) float
D) array

14. Which is a derived data type?

A) int
B) char
C) pointer
D) float

15. Structure belongs to:

A) Primary data type
B) Derived data type
C) User-defined data type
D) Modifier

16. Which data type is used to store decimal values?

A) int
B) char
C) float
D) array

17. Which data type stores single character?

A) int
B) float
C) char
D) double

18. Which data type has no value?

A) int
B) float
C) void
D) char

19. Which of the following is a constant?

A) x
B) 10
C) int
D) +

20. Which of the following is an operator?

A) int
B) +
C) a
D) 10

21. Which of the following is a special symbol?

A) int
B) a
C) ;
D) 10

22. Which of the following is a keyword?

A) sum
B) main
C) int
D) total

23. Which of the following is correct combination of tokens?

A) Keyword + Identifier + Operator
B) Loop + Function
C) Variable + Program
D) None

24. Which is NOT allowed in identifier?

A) _
B) digit
C) space
D) letter

25. Which of the following is both keyword and data type?

A) int
B) sum
C) main
D) value

26. Which of the following is not a valid token?

A) 123
B) #include

C) main
D) @var

27. What is a variable?
28. Why is &age used in scanf()?
29. What happens if a variable is not initialized?
30. Can the value of a variable be changed?
31. What is the difference between declaration and initialization?
32. Which format specifier is used for int, float, and char variables?
33. Why is stdio.h required?
34. What happens if & is omitted in scanf()?
35. Why does printf() not require &?
36. Can scanf() read character data?
37. What is the difference between %f and %lf?
38. What does %s represent?
39. How many values can printf() print in a single statement?
40. How many values can scanf() read in a single statement?
41. Why are format specifiers necessary?
42. Which function is executed first: printf() or scanf()?

Code Snippet Question:

1. **What is the output?**

```
int a = 5;

printf("%d", a++);
```

2. **What is the output?**

```
int a = 5;

printf("%d", ++a);
```

3. **What is the output?**

```
int a = 10;

printf("%d", a--);
```

4. **What is the output?**

```
int a = 10;
printf(“%d", --a);
```

5. **What is the output?**

```
int a = 10;
printf("%d", a++ + 5);
```

6. **What is the output?**

```
int a = 10;
printf("%d", ++a + 5);
```

7. **What is the output?**

```
int a = 10;
printf("%d", a += 5);
```

8. **What is the output?**

```
int a = 20;
printf("%d", a -= 5);
```

9. **What is the output?**

```
int a = 10;
```

```
printf("%d", a *= 2);
```

10. What is the output?

```
int a = 20;
printf(“%d", a /= 4);
```

11. What is the output?

```
printf("%d", 10 > 5);
```

12. What is the output?

```
printf("%d", 10 < 5);
```

13. What is the output?

```
printf("%d", 10 == 10);
```

14. What is the output?

```
printf("%d", 10 != 10);
```

15. What is the output?

```
printf("%d", 10 >= 10);
```

16. What is the output?

```
printf("%d", 5 && 10);
```

17. What is the output?

```
printf("%d", 0 && 10);
```

18. What is the output?

```
printf("%d", 5 || 0);
```

19. What is the output?

```
printf("%d", 0 || 0);
```

20. What is the output?

```
printf("%d", !0);
```

21. What is the output?

```
printf("%d", !10);
```

22. What is the output?

```
printf("%d", !!10);
```

23. What is the output?

```
printf("%d", 5 & 3);
```

24. W**hat is the output?**

```
printf("%d", 5 | 3);
```

25. What is the output?

```
printf("%d", 5 ^ 3);
```

26. What is the output?

```
printf("%d", ~5);
```

27. What is the output?

```
printf("%d", 5 << 1);
```

28. What is the output?

```
printf("%d", 5 << 2);
```

29. What is the output?

```
printf("%d", 20 >> 1);
```

30. What is the output?

```
printf("%d", 20 >> 2);
```

31. What is the output?

```
printf("%d", -8 >> 1);
```

32. What is the output?

```
printf("%d", sizeof(char));
```

33. What is the output?

```
printf("%d", sizeof(int));
```

34. What is the output?

```
int a = 5;
printf("%d", sizeof(a));
```

35. What is the output?

```
int a = 5;
printf("%d", sizeof(a++));
```

36. What is the output?

```
printf("%d", 5 + 3 * 2);
```

37. What is the output?

```
printf("%d", (5 + 3) * 2);
```

38. What is the output?

```
printf("%d", 20 / 5 * 2);
```

39. What is the output?

```
printf("%d", 20 % 3);
```

40. What is the output?

```
printf("%d", 7 % 4 + 2);
```

41. What is the output?

```
int a = 10;
printf("%d", a > 5 ? 100 : 200);
```

42. What is the output?

```
int a = 2;
printf("%d", a > 5 ? 100 : 200);
```

43. What is the output?

```
int a = 5;
printf("%d", a = 10);
```

44. What is the output?

```
int a = 5;
printf("%d", a == 10);
```

45. What is the output?

```
if(5)
  printf("Hello");
else
  printf("World");
```

46. What is the output?

```
if(0)
  printf("Hello");
else
  printf("World");
```

47. What is the output?

```
int a = 10;
printf("%d", a+++5);
```

48. What is the output?

```
int a = 5;
printf("%d", a & 1);
```

49. What is the output?

```
int a = 8;
printf("%d", a & 1);
```

50. What is the output?

```
printf("%d", 25 ^ 25);
```

51. Find the binary representation of:

A. 5
B. 10
C. 15
D. 25
E. 50

52. Find the 2's complement representation of:

A. -5
B. -10
C. -15
D. -25

53. Trace the value of variables after execution:

```
int a = 5;

int b;

b = ++a;
```

54. Trace the value of variables after execution:

```
int a = 5;

int b;

b = a++;
```

55. Trace the value of variables after execution:

```
int a = 10;

int b;

b = --a;
```

56. Trace the value of variables after execution

```
int a = 10;

int b;

b = a--;
```

57. What is the output?

```
int a = 10;

a += 5;

a *= 2;

printf("%d", a);
```

58. What is the output?

```
int a = 32;

a >>= 2;

printf("%d", a);
```

59. What is the output?

```
int a = 5;

a <<= 3;
```

```
printf(“%d", a);
```

60. What is the output?

```
printf("%d", ~(-1));
```

Programming Questions

1. Write a program to display your name using printf().
2. Write a program to read an integer using scanf() and display it using printf().
3. Write a program to read two integers and display their sum.
4. Write a program to read a character and display it.
5. Write a program to read a float value and display it.
6. Write a program to read a student's age and display it.
7. Write a program to read and display three integer values using a single scanf() statement.

Bitwise Programming Questions

1. Write a program to find the bitwise AND of two numbers.
2. Write a program to find the bitwise OR of two numbers.
3. Write a program to find the bitwise XOR of two numbers.
4. Write a program to find the bitwise complement of a number.
5. Write a program to perform left shift operation on a number.
6. Write a program to perform right shift operation on a number.
7. Write a program to check whether a number is even or odd using bitwise operations.
8. Write a program to check whether a number is even or odd without using %.
9. Write a program to check whether the least significant bit (LSB) is set or clear.
10. Write a program to multiply a number by 2 using bitwise operations.
11. Write a program to multiply a number by 4 using bitwise operations.
12. Write a program to multiply a number by 8 using bitwise operations.
13. Write a program to multiply a number by 16 using bitwise operations.
14. Write a program to divide a number by 2 using bitwise operations.
15. Write a program to divide a number by 4 using bitwise operations.
16. Write a program to divide a number by 8 using bitwise operations.
17. Write a program to divide a number by 16 using bitwise operations.
18. Write a program to check whether the nth bit is set or clear.
19. Write a program to set the nth bit of a number.
20. Write a program to clear the nth bit of a number.
21. Write a program to toggle the nth bit of a number.

22. Write a program to count the number of set bits in a number.
23. Write a program to count the number of clear bits in an 8-bit number.
24. Write a program to find the position of the least significant set bit.
25. Write a program to find the position of the most significant set bit.
26. Write a program to swap two numbers without using a third variable.
27. Write a program to swap two numbers using XOR.
28. Write a program to check whether a number is a power of 2.
29. Write a program to check whether a number is a power of 4.
30. Write a program to check whether a number contains only one set bit.
31. Write a program to determine whether two numbers have opposite signs.
32. Write a program to find the absolute value of a number using bitwise operations.
33. Write a program to reverse all bits of an 8-bit number.
34. Write a program to reverse all bits of a 16-bit number.
35. Write a program to reverse all bits of a 32-bit number.
36. Write a program to check whether the binary representation of a number is a palindrome.
37. Write a program to count consecutive set bits in a number.
38. Write a program to count consecutive clear bits in a number.
39. Write a program to find the parity of a number.
40. Write a program to find the Hamming distance between two numbers.
41. Write a program to find the unique element in an array where every other element appears twice.
42. Write a program to find two unique elements in an array where every other element appears twice.
43. Write a program to find the missing number in a sequence using XOR.
44. Write a program to check whether all bits of a number are set.
45. Write a program to toggle all bits of a number.
46. Write a program to rotate bits left by one position.
47. Write a program to rotate bits right by one position.
48. Write a program to rotate bits left by n positions.
49. Write a program to rotate bits right by n positions.
50. Write a program to extract a specific bit field from a number.
51. Write a program to insert a value into specific bit positions of a number.
52. Write a program to create a bit mask for setting specific bits.
53. Write a program to create a bit mask for clearing specific bits.
54. Write a program to set multiple bits of a number.
55. Write a program to clear multiple bits of a number.
56. Write a program to toggle multiple bits of a number.
57. Write a program to find the largest power of 2 less than or equal to a given number.
58. Write a program to find the first set bit from the right.
59. Write a program to find the first set bit from the left.
60. Write a program to determine whether a number is divisible by 2 using bitwise operations.
61. Write a program to determine whether a number is divisible by 4 using bitwise operations.

62. Write a program to determine whether a number is divisible by 8 using bitwise operations.
63. Write a program to count the total number of bits required to represent a number.
64. Write a program to convert a decimal number into binary using bitwise operations.
65. Write a program to display the binary representation of a number.
66. Write a program to compare two numbers without using relational operators.
67. Write a program to find the maximum of two numbers using bitwise operations.
68. Write a program to find the minimum of two numbers using bitwise operations.
69. Write a program to check whether a specific bit pattern exists in a number.
70. Write a program to implement a simple permission system using bit masks.
71. Write a program to set, clear, and toggle bits of a microcontroller port register.
72. Write a program to extract bits 4 to 7 from an 8-bit register.
73. Write a program to replace bits 2 to 5 of a register with a new value.
74. Write a program to count leading zeros in a number.
75. Write a program to count trailing zeros in a number.
76. Write a program to count leading ones in a number.
77. Write a program to count trailing ones in a number.
78. Write a program to determine whether a number is positive or negative using bitwise operations.
79. Write a program to multiply a number by 32 using bitwise operations.
80. Write a program to divide a number by 32 using bitwise operations.